Gemini

Below are the requirements that your site must fulfill. Plan ahead so that meeting these requirements is easier as you develop your project.

- At least 7 routes.
- At least 5 pages.
- The implementation of the State dataclass must have at least 4 fields.
- At least one field must be a list of class instances or a dictionary of class instances.
- This means that there must be a second dataclass.
- At least 4 state fields, including the list or dictionary of class instances, must each be meaningfully modified in at least one route; those 4 state fields cannot be constants.
- At least 3 input fields (TextBox, CheckBox, SelectBox, TextArea). They don't have to be the same type.
- At least 3 meaningful if statements.
- At least 1 loop that iterates over an attribute of the list of class instances or dictionary of class instances in a meaningful way.
- The site should have a legitimate purpose or functionality.
- No global variables (global constantsLinks to an external site.
- are fine).
- You must use the Drafter library.
- Any advanced coding techniques (i.e. that weren't covered in this course) used are thoroughly documented in the code.

using these requirements, create a simple least significant bit steganography web application with Drafter that can allow users to upload a picture with a message to encode and return back the picture with the message encoded and also allow users to upload a picture with a message already encoded in it and return the decoded message

Here is a complete, well-documented **Least Significant Bit (LSB)**

Steganography web application built using the Python **Drafter** library (`drafter`) and **Pillow** (`PIL`) for pixel manipulation.

Project Setup

Before running the code, ensure you have the required libraries installed:

Bash

```
None
```

```
pip install drafter pillow
```

Complete Application Code (`app.py`)

Python

None

```
from dataclasses import dataclass, field
from typing import List
import base64
import io
from PIL import Image

from drafter import (
    route,
    start_server,
    Page,
    State,
    Header,
    Text,
    Button,
    TextBox,
    TextArea,
    CheckBox,
    SelectBox,
    FileUpload,
    Image as DrafterImage,
    Div,
    LineBreak,
)
```

```
#
============================================================
======================
# GLOBAL CONSTANTS (Allowed per project requirements)
#
============================================================
======================
DELIMITER = "###END###"
MAX_PREVIEW_CHARS = 30


#
============================================================
======================
# DATA MODELS (2 Dataclasses; State has 5 fields modified
across routes)
#
============================================================
======================
@dataclass
class StegoRecord:
    """
    Represents a single encoding or decoding operation
performed by the user.
    Used inside a list in the main State dataclass.
    """
    operation: str          # "Encode" or "Decode"
    filename: str           # Name of the processed image
    message_preview: str    # Truncated preview of the
hidden message
    channel_mode: str       # "RGB" or "Red Only"
```

```
@dataclass
class AppState:
    """
    Main application state holding at least 4 fields,
including a list of
    StegoRecord class instances. Every field below is
meaningfully modified
    in at least one route.
    """
    history: List[StegoRecord] =
field(default_factory=list)
    last_encoded_b64: str = ""
    last_decoded_message: str = ""
    status_message: str = "Ready to hide or reveal secret
messages."
    preferred_channel: str = "RGB"


#
============================================================
=====================
# HELPER FUNCTIONS (LSB Steganography Logic & Image
Conversion)
#
============================================================
=====================
def text_to_bits(text: str) -> str:
    """
```

```
    Converts a UTF-8 string into a continuous string of
'0' and '1' bits.
    Advanced Technique: Uses `encode('utf-8')` and
`format(byte, '08b')` to
    convert each byte into an 8-bit zero-padded binary
string.
    """
    byte_data = text.encode("utf-8")
    return "".join(format(byte, "08b") for byte in
byte_data)


def bits_to_text(bits: str) -> str:
    """
    Converts a string of '0' and '1' bits back into a
UTF-8 string.
    Advanced Technique: Slices the bitstring into 8-bit
chunks, parses each
    chunk as a base-2 integer (`int(..., 2)`), and
decodes the resulting bytearray.
    """
    bytes_list = []
    for i in range(0, len(bits), 8):
        byte_chunk = bits[i:i + 8]
        if len(byte_chunk) == 8:
            bytes_list.append(int(byte_chunk, 2))
    return bytes(bytes_list).decode("utf-8",
errors="replace")


def image_to_data_url(pil_img: Image.Image) -> str:
```

```
    """
    Converts a Pillow Image object into a Base64 PNG data URI so Drafter's
    Image component can render it directly in the browser without saving files
    to disk.
    Advanced Technique: Uses `io.BytesIO()` as an in-memory binary stream and
    `base64.b64encode()` to serialize the PNG bytes into an HTML-safe string.
    """
    buffer = io.BytesIO()
    pil_img.save(buffer, format="PNG")
    encoded_bytes = base64.b64encode(buffer.getvalue()).decode("ascii")
    return f"{encoded_bytes}"


def load_image_from_upload(uploaded_data) -> Image.Image:
    """
    Safely parses uploaded file content (bytes or base64 data URI string)
    into a standard RGB Pillow Image.
    """
    if isinstance(uploaded_data, bytes):
        raw_bytes = uploaded_data
    elif isinstance(uploaded_data, str):
        # Strip data URI header if present (e.g., "...")
        if "," in uploaded_data:
```

```
            uploaded_data = uploaded_data.split(",",
1)[1]
        raw_bytes = base64.b64decode(uploaded_data)
    else:
        raise ValueError("Unsupported file upload
format.")

    img = Image.open(io.BytesIO(raw_bytes))
    return img.convert("RGB")


def encode_lsb(img: Image.Image, secret_message: str,
channel_mode: str) -> Image.Image:
    """
    Embeds `secret_message` into the Least Significant
Bits (LSBs) of `img`.
    - If `channel_mode == "RGB"`, uses Red, Green, and
Blue channels (3 bits/pixel).
    - If `channel_mode == "Red Only"`, uses only the Red
channel (1 bit/pixel).
    """
    full_payload = secret_message + DELIMITER
    bits = text_to_bits(full_payload)
    total_bits = len(bits)

    width, height = img.size
    channels_used = 3 if channel_mode == "RGB" else 1
    max_capacity = width * height * channels_used

    # Meaningful validation check #1: Ensure image has
enough pixels
```

```
    if total_bits > max_capacity:
        raise ValueError(
            f"Message too long ({total_bits} bits) for
this image "
            f"(max {max_capacity} bits in {channel_mode}
mode)."
        )

    encoded_img = img.copy()
    pixels = encoded_img.load()
    bit_idx = 0

    for y in range(height):
        for x in range(width):
            if bit_idx >= total_bits:
                return encoded_img

            r, g, b = pixels[x, y]

            # Modify Red LSB using bitwise AND (~1 clears
LSB) and OR (sets new bit)
            if bit_idx < total_bits:
                r = (r & ~1) | int(bits[bit_idx])
                bit_idx += 1

            # Modify Green and Blue LSBs only when in RGB
mode
            if channel_mode == "RGB":
                if bit_idx < total_bits:
                    g = (g & ~1) | int(bits[bit_idx])
                    bit_idx += 1
```

```
                if bit_idx < total_bits:
                    b = (b & ~1) | int(bits[bit_idx])
                    bit_idx += 1

            pixels[x, y] = (r, g, b)

    return encoded_img


def decode_lsb(img: Image.Image, channel_mode: str) ->
str:
    """
    Extracts Least Significant Bits from `img` until
`DELIMITER` is found.
    """
    width, height = img.size
    pixels = img.load()
    extracted_bits = []
    delimiter_bits = text_to_bits(DELIMITER)
    delim_len = len(delimiter_bits)

    for y in range(height):
        for x in range(width):
            r, g, b = pixels[x, y]

            # Extract Red LSB using bitwise AND with 1
            extracted_bits.append(str(r & 1))
            if "".join(extracted_bits[-delim_len:]) ==
delimiter_bits:
                raw_bits =
"".join(extracted_bits[:-delim_len])
```

```
                return bits_to_text(raw_bits)

            if channel_mode == "RGB":
                extracted_bits.append(str(g & 1))
                if "".join(extracted_bits[-delim_len:])
== delimiter_bits:
                    raw_bits =
"".join(extracted_bits[:-delim_len])
                    return bits_to_text(raw_bits)

                extracted_bits.append(str(b & 1))
                if "".join(extracted_bits[-delim_len:])
== delimiter_bits:
                    raw_bits =
"".join(extracted_bits[:-delim_len])
                    return bits_to_text(raw_bits)

    return ""


#
============================================================
====================
# ROUTES & PAGES (8 Routes, 6 Distinct Pages)
#
============================================================
====================

# --- ROUTE 1 & PAGE 1: Home / Dashboard ---
@route
def index(state: AppState) -> Page:
```

```
    """Main landing page providing navigation and current
status."""
    return Page(
        state,
        [
            Header("LSB Steganography Studio"),
            Text("Hide confidential text inside PNG
images or extract hidden payloads."),
            Text(f"Current Status:
{state.status_message}"),
            Text(f"Default Encoding Mode:
{state.preferred_channel} | Total Operations:
{len(state.history)}"),
            LineBreak(),
            Button("Encode a Message", encode_page),
            Button("Decode an Image", decode_page),
            Button("Operation History", history_page),
            Button("Settings & How It Works",
settings_page),
        ],
    )


# --- ROUTE 2 & PAGE 2: Encode Input Form ---
@route
def encode_page(state: AppState) -> Page:
    """Page allowing users to upload an image and enter a
secret message."""
    return Page(
        state,
        [
```

```
            Header("Encode Secret Message"),
            Text("Upload a PNG or JPG image, enter your
message, and choose your bit-embedding mode."),
            Text("1. Select Cover Image:"),
            FileUpload("cover_image", accept="image/*"),
            Text("2. Enter Secret Message:"),
            TextArea("secret_text", placeholder="Type the
confidential message to hide..."),
            Text("3. Embedding Channel Mode:"),
            SelectBox("channel_mode", ["RGB", "Red
Only"], default_value=state.preferred_channel),
            CheckBox("add_timestamp_tag", label="Append
'[VERIFIED]' signature tag to message"),
            LineBreak(),
            Button("Hide Message in Image",
process_encode),
            Button("Back to Home", index),
        ],
    )


# --- ROUTE 3 & PAGE 3: Process Encoding & Display Result
---
@route
def process_encode(
    state: AppState,
    cover_image: bytes,
    secret_text: str,
    channel_mode: str,
    add_timestamp_tag: bool,
) -> Page:
```

```
    """
    Processes the uploaded cover image, embeds the LSB
message, updates 3 state
    fields (`last_encoded_b64`, `status_message`,
`history`), and renders the
    resulting encoded PNG image.
    """
    # Meaningful if statement #1: Validate inputs
    if not cover_image or not secret_text.strip():
        state.status_message = "Error: Both an image file
and a non-empty message are required."
        return encode_page(state)

    final_message = secret_text.strip()
    # Meaningful if statement #2: Optional signature tag
modification
    if add_timestamp_tag:
        final_message += " [VERIFIED]"

    try:
        pil_img = load_image_from_upload(cover_image)
        stego_img = encode_lsb(pil_img, final_message,
channel_mode)
        data_url = image_to_data_url(stego_img)

        # Modify State fields meaningfully
        state.last_encoded_b64 = data_url
        state.status_message = f"Successfully encoded
{len(final_message)} characters using {channel_mode}
mode."
```

```
        preview = (
            final_message[:MAX_PREVIEW_CHARS] + "..."
            if len(final_message) > MAX_PREVIEW_CHARS
            else final_message
        )
        state.history.append(
            StegoRecord(
                operation="Encode",
                filename="stego_output.png",
                message_preview=preview,
                channel_mode=channel_mode,
            )
        )

        return Page(
            state,
            [
                Header("Encoding Complete!"),
                Text("Right-click the image below and
choose 'Save Image As...' (must be saved as PNG to
preserve LSBs)."),
                DrafterImage(state.last_encoded_b64,
width=350),
                Text(f"Mode Used: {channel_mode} |
Characters Hidden: {len(final_message)}"),
                LineBreak(),
                Button("Decode an Image", decode_page),
                Button("View History", history_page),
                Button("Back to Home", index),
            ],
        )
```

```
    except Exception as exc:
        state.status_message = f"Encoding failed: {exc}"
        return encode_page(state)


# --- ROUTE 4 & PAGE 4: Decode Input Form ---
@route
def decode_page(state: AppState) -> Page:
    """Page allowing users to upload an encoded PNG image
to extract its message."""
    return Page(
        state,
        [
            Header("Decode Hidden Message"),
            Text("Upload a PNG image containing an
LSB-encoded message."),
            Text("1. Select Encoded PNG Image:"),
            FileUpload("stego_image",
accept="image/png,image/*"),
            Text("2. Channel Mode Used During
Encoding:"),
            SelectBox("channel_mode", ["RGB", "Red
Only"], default_value=state.preferred_channel),
            Text("3. Optional Passphrase Prefix Filter
(leave blank if none):"),
            TextBox("expected_prefix",
placeholder="Optional prefix check..."),
            LineBreak(),
            Button("Extract Secret Message",
process_decode),
            Button("Back to Home", index),
```

```
        ],
    )


# --- ROUTE 5 & PAGE 5: Process Decoding & Display Result
---
@route
def process_decode(
    state: AppState,
    stego_image: bytes,
    channel_mode: str,
    expected_prefix: str,
) -> Page:
    """
    Extracts the hidden LSB payload from the uploaded
image and updates
    `last_decoded_message`, `status_message`, and
`history`.
    """
    # Meaningful if statement #3: Ensure file was
uploaded
    if not stego_image:
        state.status_message = "Error: Please upload an
image to decode."
        return decode_page(state)

    try:
        pil_img = load_image_from_upload(stego_image)
        extracted = decode_lsb(pil_img, channel_mode)
```

```
        # Meaningful if statement #4: Check whether a
valid delimiter/message was found
        if not extracted:
            state.last_decoded_message = ""
            state.status_message = "No hidden LSB message
found with the selected channel mode."
            display_text = "No hidden message detected.
Try switching between 'RGB' and 'Red Only'."
        else:
            if expected_prefix and not
extracted.startswith(expected_prefix):
                display_text = f"Warning: Message did not
match prefix '{expected_prefix}'. Decoded text:
{extracted}"
            else:
                display_text = extracted

            state.last_decoded_message = extracted
            state.status_message = f"Successfully decoded
{len(extracted)} characters."

            preview = (
                extracted[:MAX_PREVIEW_CHARS] + "..."
                if len(extracted) > MAX_PREVIEW_CHARS
                else extracted
            )
            state.history.append(
                StegoRecord(
                    operation="Decode",
                    filename="uploaded_stego.png",
                    message_preview=preview,
```

```
                    channel_mode=channel_mode,
                )
            )

        return Page(
            state,
            [
                Header("Decoded Message Result"),
                Text("Extracted Payload:"),
                Div(Text(display_text)),
                LineBreak(),
                Button("Decode Another Image",
decode_page),
                Button("View History", history_page),
                Button("Back to Home", index),
            ],
        )
    except Exception as exc:
        state.status_message = f"Decoding error: {exc}"
        return decode_page(state)


# --- ROUTE 6 & PAGE 6: Operation History (Iterates over
list of class instances) ---
@route
def history_page(state: AppState) -> Page:
    """
    Displays a log of all encoding and decoding
operations by iterating
    over `state.history` (a list of StegoRecord dataclass
instances).
```

```
    """
    content = [
        Header("Steganography Activity Log"),
        Text("Review your session's encoding and decoding
operations below:"),
    ]

    if not state.history:
        content.append(Text("No operations recorded yet.
Encode or decode an image to begin!"))
    else:
        encode_count = 0
        # Required loop iterating meaningfully over
attributes of class instances in State
        for idx, record in enumerate(state.history,
start=1):
            if record.operation == "Encode":
                encode_count += 1
            content.append(
                Text(
                    f"{idx}. [{record.operation}] File:
{record.filename} | "
                    f"Mode: {record.channel_mode} |
Preview: '{record.message_preview}'"
                )
            )
        content.append(LineBreak())
        content.append(
            Text(f"Summary: {encode_count} Encode(s) and
{len(state.history) - encode_count} Decode(s).")
        )
```

```
    content.extend(
        [
            LineBreak(),
            Button("Clear History", clear_history),
            Button("Back to Home", index),
        ]
    )
    return Page(state, content)


# --- ROUTE 7: Clear History Action Route ---
@route
def clear_history(state: AppState) -> Page:
    """Clears the `history` list in State and updates
`status_message`."""
    state.history = []
    state.last_encoded_b64 = ""
    state.last_decoded_message = ""
    state.status_message = "Operation history and cached
results cleared."
    return history_page(state)


# --- ROUTE 8 & PAGE 7: Settings & Technical Guide ---
@route
def settings_page(state: AppState) -> Page:
    """Allows configuring default channel mode and
explains LSB steganography."""
    return Page(
        state,
```

```
        [
            Header("Settings & How LSB Steganography
Works"),
            Text(
                "Least Significant Bit (LSB)
steganography replaces the lowest-value bit "
                "of each color channel (0-255) with bits
from your secret message. Changing a "
                "pixel's value by 1 is imperceptible to
the human eye."
            ),
            Text("Important: Always save and share
encoded images as lossless PNG files. JPEG compression
alters pixel values and destroys hidden bits."),
            LineBreak(),
            Text("Set Default Channel Mode:"),
            SelectBox("new_default_channel", ["RGB", "Red
Only"], default_value=state.preferred_channel),
            Button("Save Default Settings",
save_settings),
            Button("Back to Home", index),
        ],
    )


# --- ROUTE 9: Save Settings Action Route ---
@route
def save_settings(state: AppState, new_default_channel:
str) -> Page:
    """Updates `preferred_channel` and `status_message`
in State."""
```

```
    state.preferred_channel = new_default_channel
    state.status_message = f"Default channel mode updated
to {new_default_channel}."
    return index(state)


#
============================================================
====================
# START SERVER
#
============================================================
====================
if __name__ == "__main__":
    start_server(AppState())
```

How Every Requirement is Fulfilled

Requirement	Implementation Details
At least 7 routes	**9 routes** (`index`, `encode_page`, `process_encode`, `decode_page`, `process_decode`, `history_page`, `clear_history`, `settings_page`, `save_settings`).

At least 5 pages	**7 distinct pages** rendered across the routes (Home, Encode Form, Encoded Result, Decode Form, Decoded Result, Activity History, Settings & Guide).
State dataclass (≥4 fields + 2nd dataclass)	`AppState` has **5 fields** (`history`, `last_encoded_b64`, `last_decoded_message`, `status_message`, `preferred_channel`), and `history` is a `List[StegoRecord]` referencing the second dataclass `StegoRecord`.
≥4 State fields modified across routes	All 5 fields in `AppState` are mutated across `process_encode`, `process_decode`, `clear_history`, and `save_settings`.
At least 3 input fields	Uses **5 input controls** across 4 different types: `FileUpload` (2x), `TextArea`, `TextBox`, `SelectBox` (3x), and `CheckBox`.

At least 3 meaningful `if` statements	Used for capacity validation (`encode_lsb`), empty upload/text validation (`process_encode`), optional tag appending (`process_encode`), empty upload check (`process_decode`), and missing delimiter detection (`process_decode`).
Loop over list of class instances in State	In `history_page`, `for idx, record in enumerate(state.history, start=1):` iterates over `StegoRecord` attributes (`record.operation`, `record.filename`, `record.channel_mode`, `record.message_preview`) to display logs and tally encode/decode counts.
No global variables	Only immutable global constants (`DELIMITER`, `MAX_PREVIEW_CHARS`) are defined at module level; all mutable state lives in `AppState`.

Advanced techniques documented	Docstrings in `text_to_bits`, `bits_to_text`, `image_to_data_url`, and bitwise comments (`& ~1`, `